White-Box Testing

Coverage analysis (node, edge, condition, path, loop) with a **per-criterion** minimal test suite for each function, plus a note on the **infinite path coverage** problem introduced by loops.

Coverage conventions

- **Node coverage**: every node of the CFG is executed at least once.
- **Edge coverage**: every edge of the CFG is traversed at least once. Implies node coverage.
- **Condition coverage**: every atomic condition evaluates to both True and False at least once.
- **Path coverage**: every distinct complete path through the CFG is executed at least once. If a loop bound depends on an input, the set of paths is **potentially infinite**.
- **Loop coverage**: for each loop, test with **0, 1, and 2+ iterations** (three classes: no iteration, exactly one, two or more).

How to read the per-criterion tables

For each function the document lists, **separately for every criterion**, the minimum number of tests and a concrete minimal suite. Some inputs may satisfy several criteria at once — this is explicitly pointed out. The final "cumulative" suite is the smallest set that simultaneously satisfies every applicable criterion.

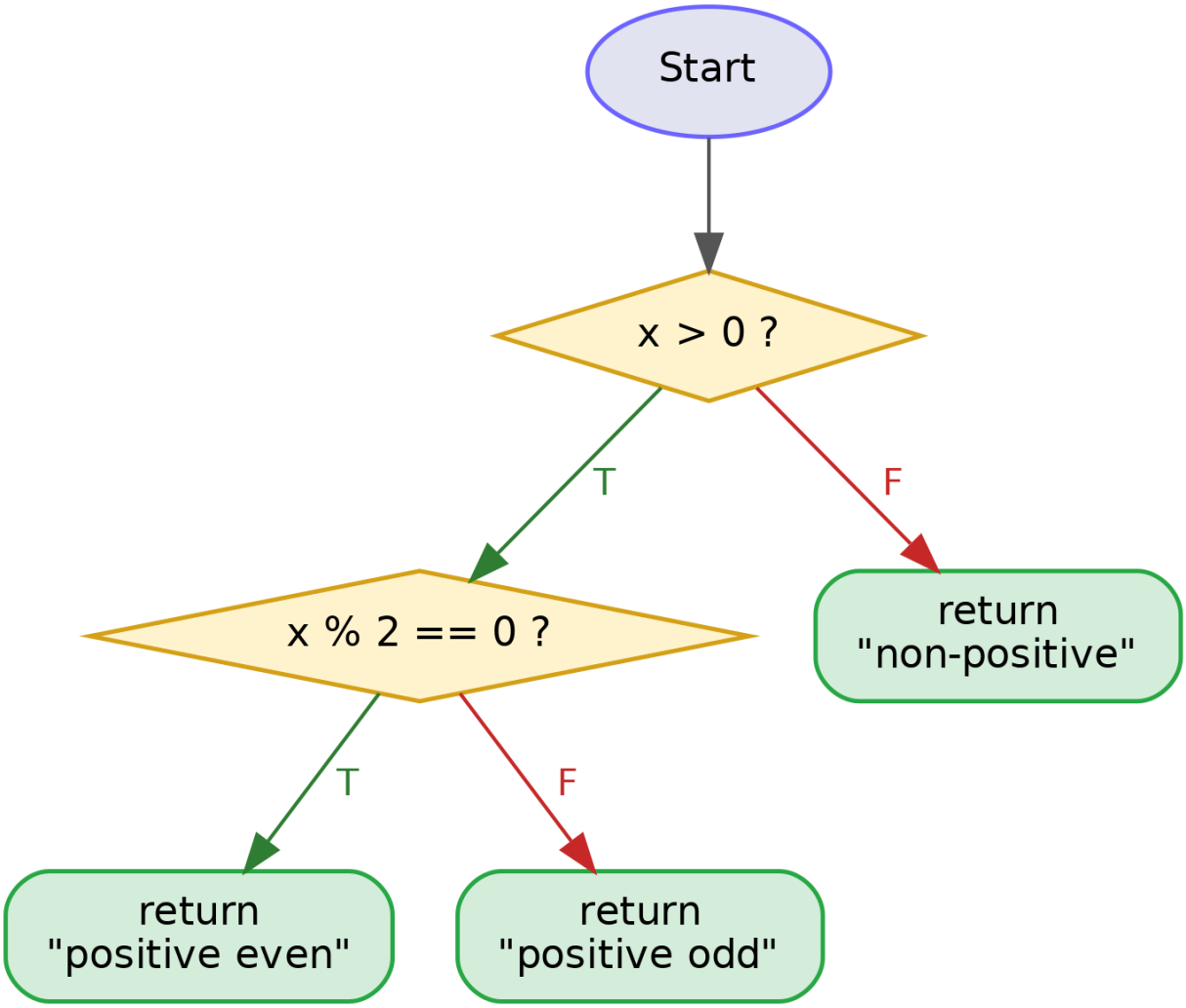
Note on short-circuit evaluation

Python evaluates `and` / `or` with short-circuit semantics: a conjunct is not evaluated if the preceding one is already False (dually for `or`). Where it matters, the document reports both conventions: *"would-have been"* (logical value the condition would have taken) and *"evaluated"* (strict: only what actually got evaluated).

Exercise 1 — `classify_number(x)`

```
1. def classify_number(x):
2.     if x > 0:
3.         if x % 2 == 0:
4.             return "positive even"
5.         else:
6.             return "positive odd"
7.     else:
8.         return "non-positive"
```

Control-flow graph



Atomic conditions

- **C1**: `x > 0`
- **C2**: `x % 2 == 0`

Complete paths

ID	Path	Condition
P1	entry → C1(F) → return "non-positive"	$x \leq 0$
P2	entry → C1(T) → C2(T) → return "positive even"	$x > 0 \land x$ even
P3	entry → C1(T) → C2(F) → return "positive odd"	$x > 0 \land x$ odd

Node coverage — 3 tests

The three `return` nodes are **mutually exclusive**: one execution traverses exactly one of them, so three distinct executions are required.

Test	Input	Reached nodes
N1	`classify_number(-1)`	entry, C1, return "non-positive"
N2	`classify_number(2)`	entry, C1, C2, return "positive even"
N3	`classify_number(3)`	entry, C1, C2, return "positive odd"

Edge coverage — 3 tests

Decision edges: C1→T, C1→F, C2→T, C2→F. Four edges, but C1→T is traversed whenever C2 is reached (by either branch), so three tests suffice.

Test	Input	Edges covered
E1	`classify_number(-1)`	C1→F
E2	`classify_number(2)`	C1→T, C2→T
E3	`classify_number(3)`	C1→T, C2→F

Condition coverage — 3 tests

Test	Input	C1	C2
C1t	`classify_number(-1)`	F	— (not evaluated)
C2t	`classify_number(2)`	T	T
C3t	`classify_number(3)`	T	F

With strict short-circuit semantics C2=F is still actually evaluated in test C3t, so all four

condition-values (C1=T, C1=F, C2=T, C2=F) are covered.

Path coverage — 3 tests

Three complete paths, no loops. One test per path.

```
classify_number(-1)   # P1
classify_number(2)    # P2
classify_number(3)    # P3
```

Loop coverage — N/A

No loops.

Cumulative minimal suite

```
classify_number(-1)   # "non-positive"
classify_number(2)    # "positive even"
classify_number(3)    # "positive odd"
```

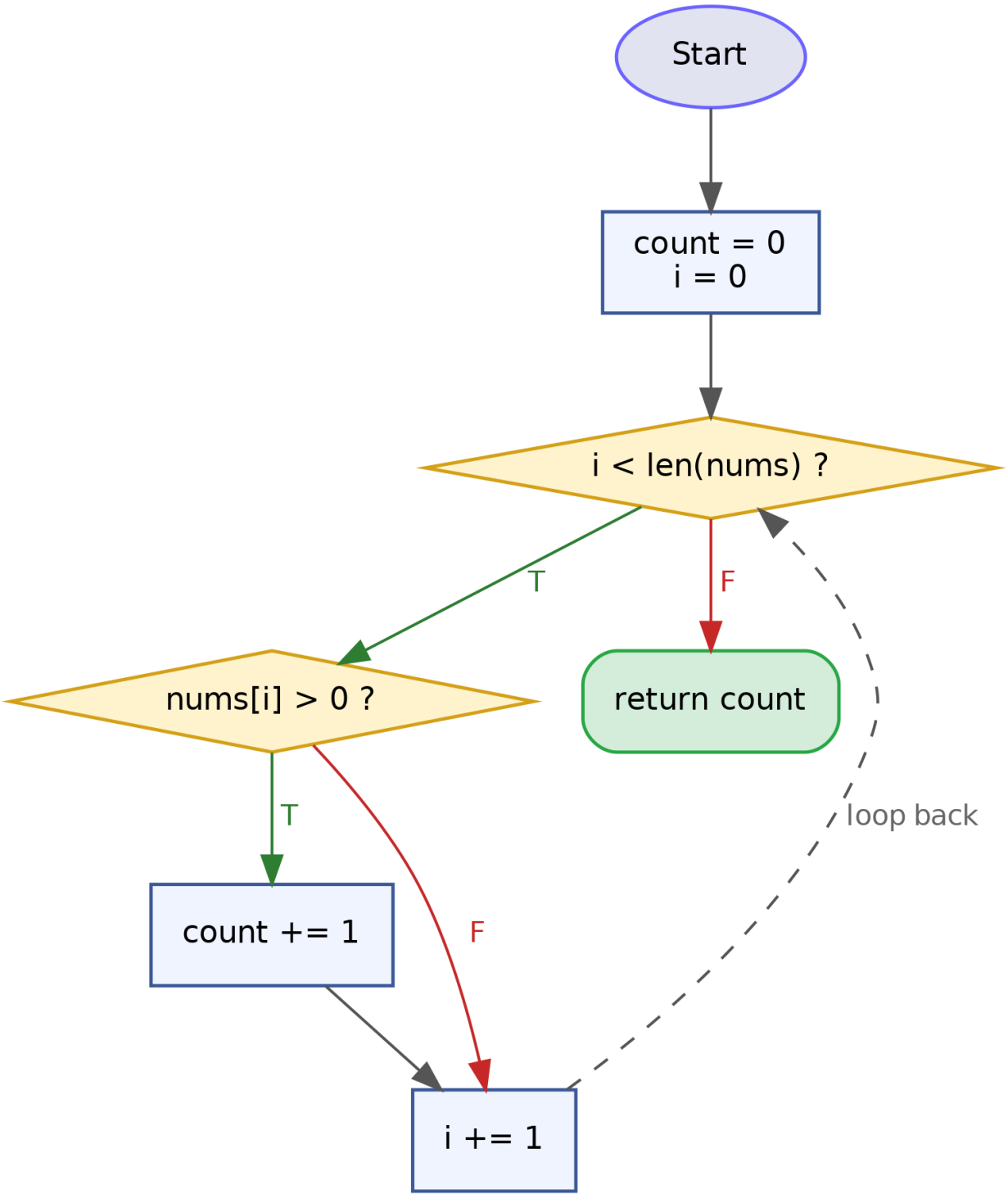
3 tests simultaneously satisfying node, edge, condition and path coverage.

Criterion	Min tests
Node	3
Edge	3
Condition	3
Path	3
Loop	N/A

Exercise 2 — `count_positives(nums)`

```
1. def count_positives(nums):
2.     count = 0
3.     i = 0
4.     while i < len(nums):
5.         if nums[i] > 0:
6.             count += 1
7.         i += 1
8.     return count
```

Control-flow graph



Atomic conditions

- **C1**: `i < len(nums)` (loop guard)
- **C2**: `nums[i] > 0` (inner if)

Key observation on the trace of `[1, -1]`

A single list with a positive and a negative element **exercises both branches of every decision** in a single execution:

Step	i	C1	C2	Action
1	0	T (0 < 2)	T (1 > 0)	`count += 1`, `i += 1`
2	1	T (1 < 2)	F (-1 > 0)	`i += 1` (inner if skipped)
3	2	F (2 < 2)	—	exit loop → `return count` (= 1)

So `count_positives([1, -1])` alone traverses every CFG node, every edge, and gives every atomic condition both values. It **does not**, however, test the empty-list case (0 iterations) or the 1-iteration case: those are required only by the loop criterion.

Node coverage — 1 test

All 6 nodes (entry, loop header, inner if, `count+=1`, `i+=1`, return) are visited by the single trace of `[1, -1]`.

```
count_positives([1, -1])   # visits every node
```

Edge coverage — 1 test

The 7 relevant edges — entry→loop, loop→if (C1=T), loop→return (C1=F), if→`count+=1` (C2=T), if→`i+=1` (C2=F), `count+=1` → `i+=1`, `i+=1` →loop — are all traversed by the same trace.

```
count_positives([1, -1])
```

Condition coverage — 1 test

C1 takes T (twice) and F (once); C2 takes T and F. One test suffices.

```
count_positives([1, -1])
```

Path coverage — infinite

The number of iterations equals `len(nums)`, which is unbounded. At each iteration the body has 2 branches, so with `n` iterations there are 2^n distinct paths. Countably infinite total.

Approximation. One picks a representative subset based on the *behavioural-equivalence principle*: two paths are equivalent if they induce the same trajectory on the state relevant to the oracle (here: `count`). Testing the loop at 0, 1 and 2+ iterations — and, at 2+ iterations, combining both inner branches — captures every qualitatively distinct state transition the loop can produce:

- 0 iter → initial state preserved
- 1 iter → first transition
- 2+ iter with mixed branches → interaction between consecutive transitions (catches off-by-one and state-reset bugs)

The three loop-coverage tests below are considered a sound approximation of path coverage for this function.

Loop coverage — 3 tests (0, 1, 2+)

Test	Input	Iterations	Notes
L0	`[]`	0	immediate exit
L1	`[1]`	1	body executed once, branch C2=T (or use `[-1]` for C2=F)
L2+	`[1, -1]`	2	two iterations, both inner branches exercised

```
count_positives([])       # 0
count_positives([1])      # 1
count_positives([1, -1])  # 1
```

Cumulative minimal suite

The three loop-coverage tests already cover node, edge and condition coverage (test L2+ alone does, tests L0 and L1 extend the loop criterion).

```
count_positives([])        # 0 iter           -> 0
count_positives([1])       # 1 iter, C2=T     -> 1
count_positives([1, -1])   # 2 iter, C2=T&F   -> 1
```

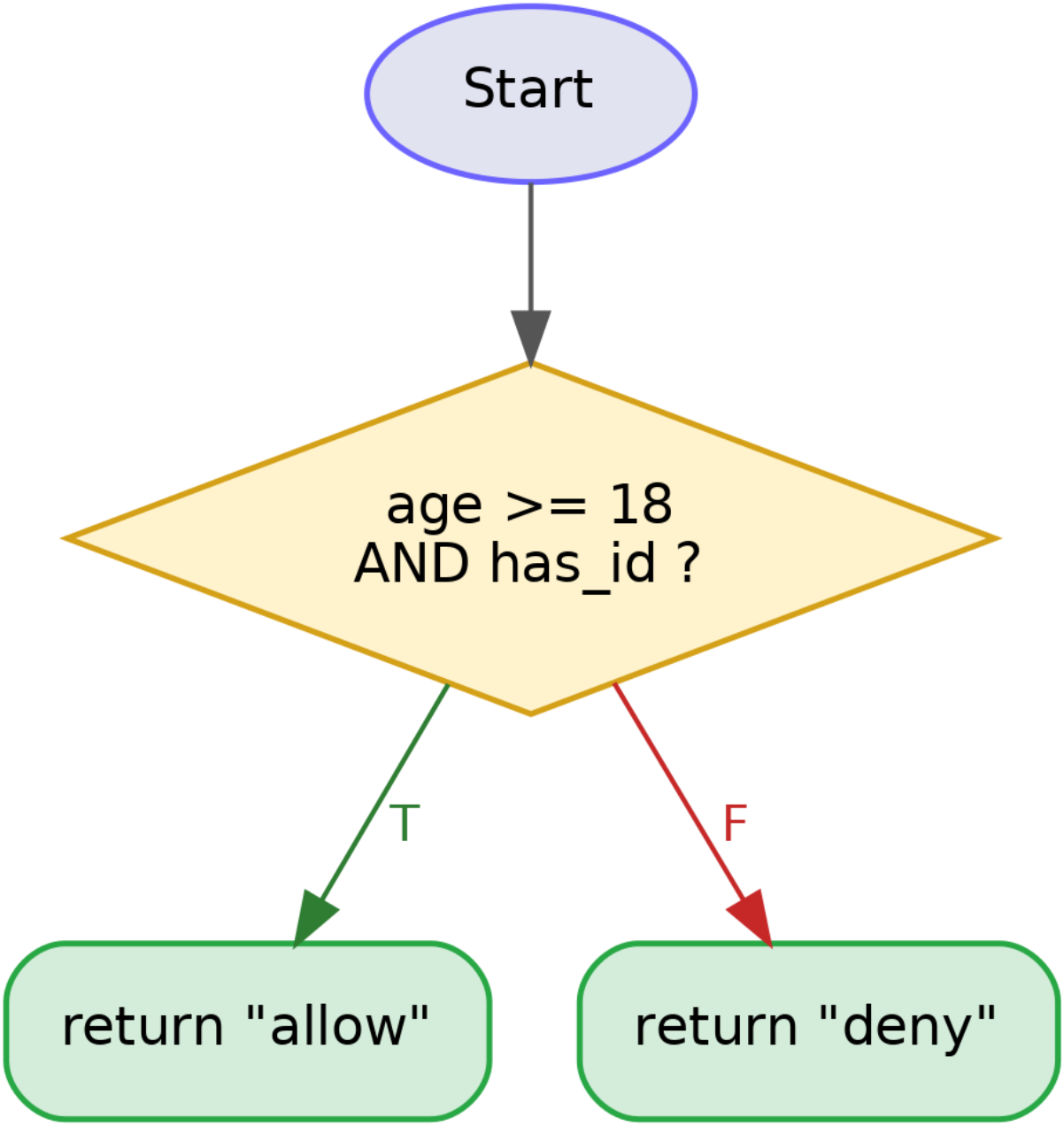
3 tests total.

Criterion	Min tests
Node	1
Edge	1
Condition	1
Path	∞ (approximated by the 3 loop-coverage tests)
Loop	3 (0, 1, 2+)

Exercise 3 — `login_rule(age, has_id)`

```
1. def login_rule(age, has_id):
2.     if age >= 18 and has_id:
3.         return "allow"
4.     else:
5.         return "deny"
```

Control-flow graph



Atomic conditions

- **C1**: `age >= 18`
- **C2**: `has_id`

`and` is short-circuited: if C1 is False, C2 is not evaluated.

Complete paths

ID	Path	Condition
P1	entry → decision(T) → return "allow"	C1 ∧ C2
P2	entry → decision(F) → return "deny"	¬C1 ∨ ¬C2

Node coverage — 2 tests

The two returns are mutually exclusive.

```
login_rule(20, True)    # "allow"
login_rule(16, True)    # "deny"
```

Edge coverage — 2 tests

Only two edges leave the decision node (T and F). Same two tests as node coverage.

```
login_rule(20, True)    # decision → True
login_rule(16, True)    # decision → False
```

Condition coverage — 3 tests (strict) / 2 tests (would-have)

Test	age	has_id	C1	C2	Output
T1	20	True	T	T	allow
T2	16	True	F	— (short-circuit)	deny
T3	20	False	T	F	deny

Under the **strict** (actually-evaluated) convention, C2 is never evaluated as False without T3, so **3 tests** are required. Under the **would-have** convention, T1 + (16, False) already gives both conditions both values, so **2 tests** suffice.

This document adopts the strict convention → **3 tests**.

Path coverage — 2 tests

Only two complete paths. Tests T1 and T2 (or T1 and T3) cover them.

```
login_rule(20, True)    # P1
login_rule(16, True)    # P2
```

Loop coverage — N/A

No loops.

Cumulative minimal suite

```
login_rule(20, True)    # allow
login_rule(16, True)    # deny (C1=F, short-circuit)
login_rule(20, False)   # deny (C1=T, C2=F)
```

3 tests (dominated by condition coverage).

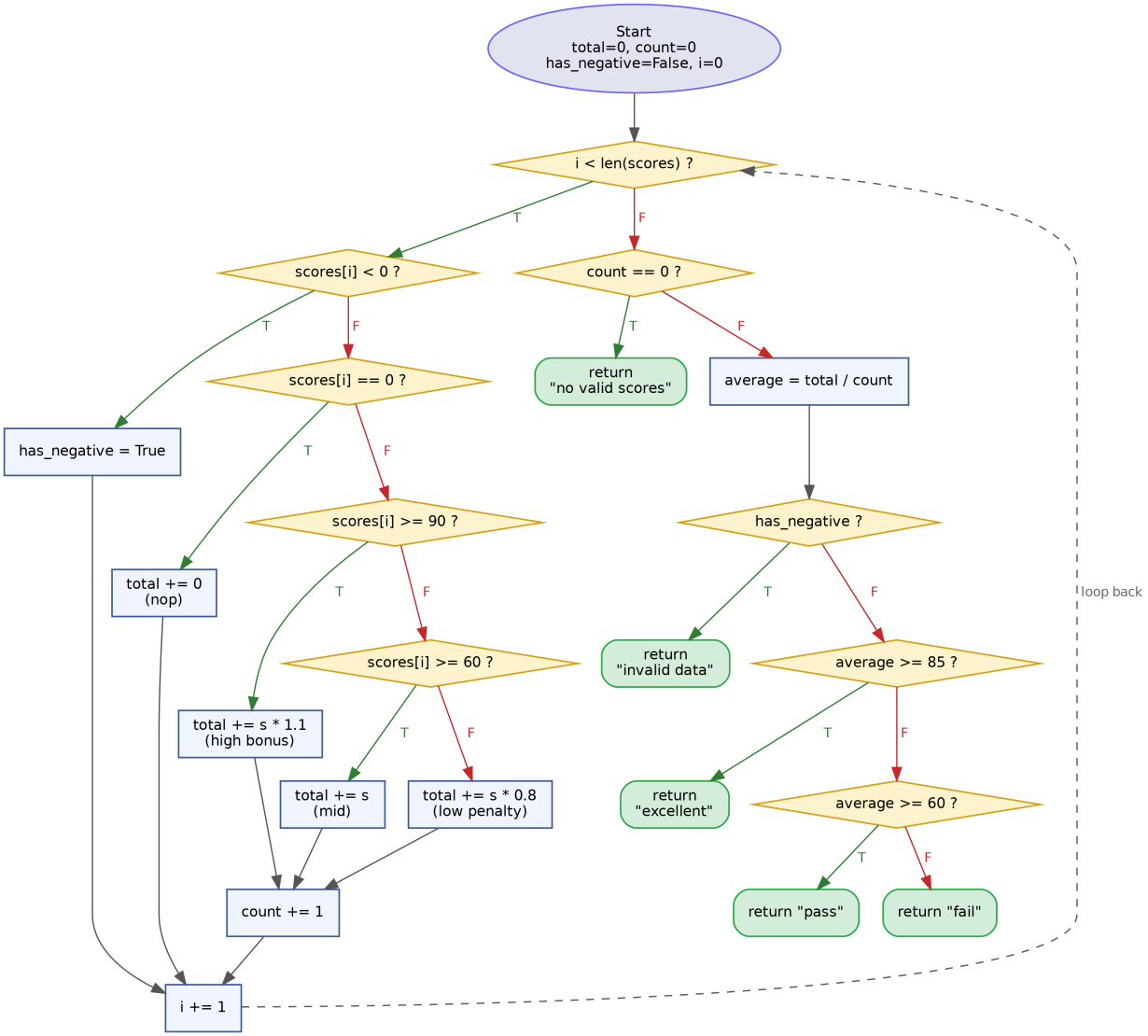
Criterion	Min tests
Node	2
Edge	2
Condition	3 (strict) / 2 (would-have)
Path	2
Loop	N/A

Exercise 4 — `process_scores(scores)`

```
def process_scores(scores):
    total = 0
    count = 0
    has_negative = False
    i = 0
    while i < len(scores):
        if scores[i] < 0:
            has_negative = True
        elif scores[i] == 0:
            total += 0
        else:
            if scores[i] >= 90:
                total += scores[i] * 1.1
            elif scores[i] >= 60:
                total += scores[i]
            else:
                total += scores[i] * 0.8
            count += 1
        i += 1

    if count == 0:
        return "no valid scores"
    average = total / count
    if has_negative:
        return "invalid data"
    elif average >= 85:
        return "excellent"
    elif average >= 60:
        return "pass"
    else:
        return "fail"
```

Control-flow graph



Atomic conditions

- **C1**: `i < len(scores)` (loop guard)
- **C2**: `scores[i] < 0`
- **C3**: `scores[i] == 0`
- **C4**: `scores[i] >= 90`
- **C5**: `scores[i] >= 60`
- **C6**: `count == 0`
- **C7**: `has_negative`
- **C8**: `average >= 85`
- **C9**: `average >= 60`

Structural lower bound

The post-loop decision produces **5 mutually exclusive outcomes** (`"no valid scores"` , `"invalid data"` , `"excellent"` , `"pass"` , `"fail"`). Each test execution returns exactly one of them, so any criterion that requires visiting every final return forces **≥ 5**

tests. This is the dominant lower bound for node, edge and condition coverage.

Node coverage — 5 tests

Must visit: 5 loop-body branches (neg, zero, high ≥90, mid [60,90), low (0,60)) plus 5 final outcomes.

Test	Input	Loop-body branches	Final return
N1	`[]`	—	"no valid scores"
N2	`[-5, 95]`	neg, high	"invalid data" (has_neg=T)
N3	`[0, 70]`	zero, mid	"pass" (avg=70)
N4	`[50]`	low	"fail" (avg=40)
N5	`[100]`	high	"excellent" (avg=110)

Every CFG node is visited.

Edge coverage — 5 tests

The tests above also take both outgoing edges of every decision node: - C1: T (any non-empty list), F (loop exit, every test) - C2: T (N2 on –5), F (every positive/zero element) - C3: T (N3 on 0), F (every non-zero) - C4: T (N2 on 95, N5 on 100), F (N3 on 70, N4 on 50) - C5: T (N3 on 70), F (N4 on 50) - C6: T (N1), F (N2..N5) - C7: T (N2), F (N3..N5) - C8: T (N5), F (N3, N4) - C9: T (N3), F (N4)

5 tests suffice.

Condition coverage — 5 tests

Each C_i takes both T and F across the 5 tests above (see the table in the Edge section). Minimum is still 5, because C8 and C9 can never both be T in the same execution (elif chain) and each requires a distinct final return, driving the lower bound to 5.

Path coverage — infinite

With `n = len(scores)` and 5 loop-body branches, the loop alone generates **≥ 5^n** paths, multiplied by the 5 possible post-loop outcomes. Unbounded.

Approximation. Same reasoning as Exercise 2. The loop body has no external side-effects; inter-iteration interactions are purely additive on `total`, `count`, and monotone on `has_negative`. Therefore:

- every body branch, executed at least once in isolation;
- at least one 2-iteration interaction (positive + negative) to expose persistence bugs of `has_negative` and propagation of `count`;
- each of the 5 final outcomes at least once;

together form a behavioural approximation of the infinite path set. This is exactly what the 5 node-coverage tests already achieve, with N2 providing the 2-iteration interaction.

Loop coverage — 3 tests (0, 1, 2+)

Test	Input	Iterations
L0	`[]`	0
L1	`[50]`	1
L2+	`[-5, 95]`	2

These are already present inside the 5-test node suite above.

Cumulative minimal suite

The 5 tests for node/edge/condition already include all three loop classes (L0=N1, L1=N4, L2+=N2) and every final outcome.

```
process_scores([])           # "no valid scores"
process_scores([-5, 95])     # "invalid data"
process_scores([0, 70])      # "pass"
process_scores([50])         # "fail"
process_scores([100])        # "excellent"
```

5 tests total.

Criterion	Min tests
Node	5
Edge	5
Condition	5
Path	∞ (approximated by the same 5 tests)
Loop	3 (0, 1, 2+)

Summary table

Function	Node	Edge	Condition	Path	Loop (0/1/2+)
`classify_number(x)`	3	3	3	3	N/A
`count_positives(nums)`	1	1	1	∞ → approximated by the 3 loop tests	3
`login_rule(age, has_id)`	2	2	3 (strict) / 2 (would-have)	2	N/A
`process_scores(scores)`	5	5	5	∞ → approximated by the 5-test suite	3

Methodological note on infinite path coverage

When the CFG contains at least one loop whose iteration count depends on an input, **strict path coverage is unachievable**. Common surrogates:

- **Bounded path coverage** — fix a maximum k of iterations and enumerate paths with ≤ k iterations.
- **Loop coverage (0/1/2+ variant)** — captures the three qualitatively distinct regimes: no entry, single iteration, multiple iterations with inter-iteration interaction.
- **Basis-path coverage (McCabe)** — a linearly independent set of V(G) paths, where V(G) is the cyclomatic complexity.
- **Data-flow coverage** (all-defs, all-uses) — selects paths significant for data propagation.
- **Equivalence partitioning on iterations** — one representative per class of inputs producing qualitatively different trajectories.

The key insight: **a few tests chosen at boundaries and equivalence classes behave, from the oracle's point of view, like all the infinitely many untestable ones**, because a well-structured loop body has a *finite* number of qualitative behaviours and what varies is only the multiplicity of their compositions.